GPW WATS 2.04 BenDec Message Definition Format

Date: 09.01.2024 | Version: GPW0.59

Fundusze Europejskie
Rzeczpospolita Polska
Unia Europejska Europejskie Fundusze Strukturalne i Inwestycyjne
NCBR Narodowe Centrum Badań i Rozwoju

CONTENTS

1. DISCLAIMER

This document is for information purposes only and does not form any part of contractual documentation. Reasonable care has been taken to ensure details contained within are accurate and not misleading at the time of publication. Warsaw Stock Exchange is not responsible for any errors or omissions contained in this document.

Warsaw Stock Exchange reserves the right to treat information contained in this document subject to later change without prior notice. This document contains confidential information to Warsaw Stock Exchange and may not be reproduced, disclosed, or used in whole or part, in any manner, without prior written consent from the owner of this document. Information included in this document shall be maintained and exercised with adequate security measures necessary to protect confidential information from unauthorized access or disclosure.

2. PREFACE

This document has been prepared by Warsaw Stock Exchange in order to help in the implementation process of WATS trading platform. This section describes the basic information about BenDec Message Definition Format. You can learn about the target audience, the document purpose and all the necessary documents you should read in relation to this Specification.

2.1. TARGET AUDIENCE

This document has been prepared to development staff, Independent Software Vendors who produce software integrated with GPW WATS, analysts, market participants and all clients who want to deepen their knowledge about GPW WATS.

2.2. DOCUMENT PURPOSE

This document contains BenDec Message format description. BenDec is a message format used to define Native Order Gateway and Market Data messages by GPW WATS.

2.3. ASSOCIATED DOCUMENTS

GPW WATS 2.04 BenDec Message Definition Format is a part of GPW WATS documentation set.

Please check the following documents to learn about the construction of Trading System.

- GPW WATS 1.01 Trading System

Please check the documentation of the trading protocols supported by GPW WATS.

- GPW WATS 2.01 Native Order Gateway Specification
- GPW WATS 2.02 FIX Order Gateway Specification .

Please check the description of the communication with Data Distribution Service.

- GPW WATS 3.01 Market Data Protocol

Please check the additional documentation which explains other services provided within GPW WATS.

- GPW WATS 4.01 Drop Copy Gateway
- GPW WATS 4.02 Post Trade Gateway
- GPW WATS 5.01 Risk Management Gateway.

Please check the additional documentation describing the following:

- GPW WATS 2.03 Rejection Codes
- GPW WATS 2.04 BenDec Message Definition Format (this document)
- GPW WATS 6.01 Connectivity.

It is recommended to read the GPW WATS 1.01 Trading System document first.

3. Document History

Version	Date	Description
0.51	29.06.2023	The initial publication of the documentation.
0.52	26.07.2023	Publication of version 0.52.
0.53	16.08.2023	Publication of version 0.53.
0.54	20.09.2023	Publication of version 0.54.
0.55	13.10.2023	Publication of version 0.55.
0.56	08.11.2023	Publication of version 0.56.
0.57	30.11.2023	Publication of version 0.57.
0.58	14.12.2023	Publication of version 0.58.
0.59	09.01.2024	Publication of version 0.59. Minor editorial changes.

4. BenDec Messages Definition Format

All binary messages used in the system are defined using special BenDec (Binary Encoder Decoder) format. It uses JSON file as the contract used to describe the message structure.

BenDec description format is used for definition of Native Order Gateway protocol (also known as binary protocol- BIN) and Market Data protocol (MD) . .

BenDec library currently supports code generators to Rust, TypeScript, Java and C++ languages (see https://github.com/fudini/bendec).

Protocol definitions are organized into 2 different files:

- md.json – includes the definition for the Market Data Protocol (MD);
- trading_port.json - includes the definition for the Native Order Gateway protocol (BIN).

4.1. Message Union

The key part of a definition is a special union Message, which enumerates all structures used as messages.

```
{
   "kind": "Union",
   "name": "Message",
   "members": [
     "Heartbeat",
 [...]
   ],
   "discriminator": [
   "header",
   "msgType"
 ]
```

As shown above, a type of the message is determined using a msgType field in the header field.

4.2. MsgType Enumeration

Each message has assigned msgType value, listed in MsgType enumeration.

```
 {
   "kind": "Enum",
   "name": "MsgType",
   "underlying": "u16",
   "variants": [
     [
       "Heartbeat",
       1,
       "A message type used to check connectivity."
     ],
 [...]
```

```
   ]
 },
Each message is described as a JSON array:
Structure name used as a message;
msgType value assigned to the message;
Message comment.
```

4.3. Primitive Types

Each primitive, the programming language defined type, is defined using Primitive convenience "kind": "Primitive" notation,

```
   "name": "u16",
   "size": 2,
   "description": "Unsigned integer, 16-bit."
 },
```

Primitive declaration has following fields:

1. kind = "Primitive";
2. name – name of primitive type;
3. size – size of the type (in bytes);
4. description – comment on the primitive.

4.4. Aliases

Alias is used for defining domain-related name to other type.

```
{
   "kind": "Alias",
   "name": "Date",
   "alias": "u32",
   "description": "Date (yyyymmdd) as integer value.\r\nIn case of undefined date value of '0' (zero) is used."
 },
```

Alias definition has following fields:

1. kind = "Alias";
2. name – name of defined alias type;
3. alias – name of aliased type;
4. description – comment on the alias;

4.5. Arrays

Array is used to define simple array of items.

```
{
   "kind": "Array",
   "name": "BICCode",
   "type": "AnsiChar",
   "length": 8,
   "description": "Business Identification Code as specified in ISO 9362."
 },
```

Array definition has following fields:

1. kind = "Array";
2. name – name of defined type;
3. type – type of items;
4. length – number of items;
5. description – comment on the array.

4.6. Enumerations

Enumerations are defined as follows:

```
{
   "kind": "Enum",
   "name": "IssueSizeType",
   "description": "Type of issue size.",
   "underlying": "u8",
   "variants": [
     [
       "Quantity",
       1,
       "Issue size expressed in quantity."
     ],
     [
       "Value",
       2,
       "Issue size expressed as nominal value."
     ],
     [
       "NoIssueSize",
       3,
       "No issue size."
     ]
   ]
 },
```

Enumeration definition uses following fields:

1. kind = "Enum";
2. name – name of enumeration;

3. description – comment on enumeration;
4. underlying – name of type that is used for values;
5. variants – array of elements:
    a) name – enumerated value name,
    b) value – value of enumerated value,
    c) comment – comment on the value.

4.7. STRUCTURES

Structure is set of fields grouped for convenience.

```
{
   "kind": "Struct",
   "name": "Header",
   "description": "Header used in Core Bus messages.",
   "fields": [
     {
       "name": "length",
       "type": "MsgLength",
       "description": "Total length of the message."
     },
 [...]
   ]
 },
```

Structure definition has following fields:

1. kind = "Struct";
2. name – name of the structure;
3. description – comment on the structure;
4. fields – array of attributes:
    a) name – name of the attribute,
    b) type – type of the attribute,
    c) description – comment on the attribute.

5. Protocol Definition - Primitives

Name	Size	Description
bool	1	Boolean
u8	1	Unsigned integer, 8-bit.
u16	2	Unsigned integer, 16-bit.
u32	4	Unsigned integer, 32-bit.
u64	8	Unsigned integer, 64-bit.
i64	8	Signed integer, 64-bit.
f64	8	A 64-bit floating point type (specifically, the binary64 type defined in IEEE 754-2008).
AnsiChar	8	